Documentação de Software

Sistema de Gerenciamento de Oficina Mecânica

Cadastro e gerenciamento de clientes, veículos e agendamentos de serviços, com autenticação de usuários e controle seguro de dados.

Stack: React (Vite) · Node.js/Express · MySQL · variáveis de ambiente (.env)

Autor: *(inserir nome completo)*

1. Visão geral

Aplicação web de três camadas. O **front-end** em React consome uma **API REST** em Node.js/Express, que persiste os dados em um banco **MySQL**. Toda configuração sensível (credenciais do banco, segredo do JWT, porta, origem permitida) fica em arquivos `.env`, fora do código-fonte.

Camada	Tecnologia	Responsabilidade
Front-end	React 18, React Router, Vite	Telas de login, painel, clientes, veículos e agendamentos
Back-end	Node.js, Express, mysql2, bcryptjs, jsonwebtoken, helmet	API REST, autenticação, validação e regras de negócio
Banco	MySQL 8 / MariaDB	Persistência com integridade referencial (InnoDB)
Configuração	dotenv (.env) e Vite env (VITE_*)	Credenciais e parâmetros fora do código

2. Requisitos

2.1 Funcionais

ID	Requisito
RF01	Autenticar usuários (e-mail e senha), criar conta e encerrar sessão.
RF02	Cadastrar, listar, buscar, editar e excluir clientes.
RF03	Cadastrar, listar, buscar, editar e excluir veículos vinculados a clientes.
RF04	Criar, listar (filtro por status), editar e excluir agendamentos de serviço.
RF05	Exibir painel com totais e os próximos serviços.

2.2 Não funcionais

ID	Requisito
RNF01	Senhas armazenadas apenas como hash bcrypt.
RNF02	Autenticação por JWT em cookie httpOnly/SameSite=Strict.
RNF03	Consultas SQL parametrizadas (prepared statements).
RNF04	Segredos e credenciais exclusivamente em variáveis de ambiente.
RNF05	Integridade referencial garantida pelo banco (chaves estrangeiras).

3. Modelagem de dados

Cinco entidades: um **cliente** possui vários **veículos**; um veículo recebe vários **agendamentos**; cada agendamento refere-se a um **serviço** do catálogo e é registrado por um **usuário**. O script `treino_db.sql` cria e popula todas as tabelas.

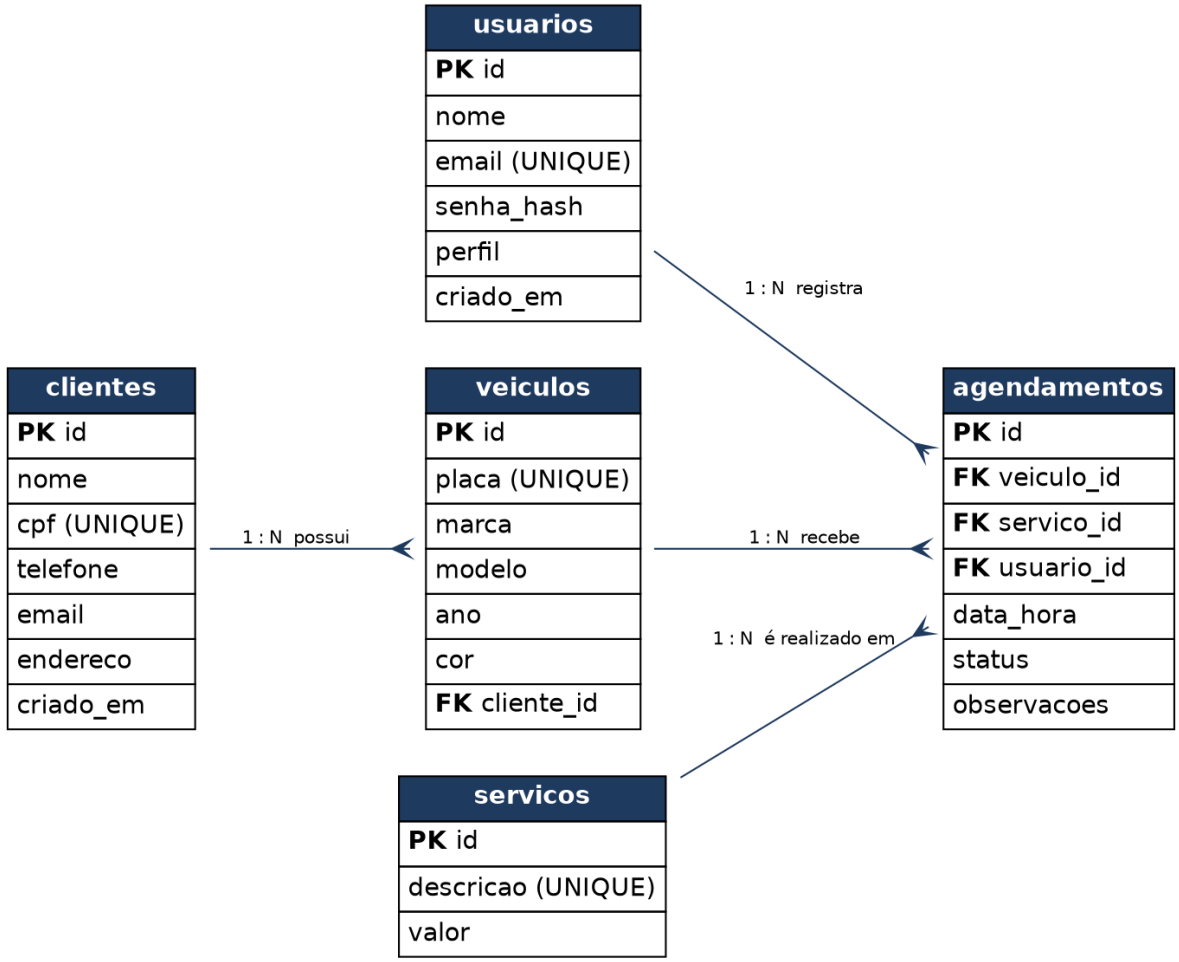


Tabela	Regras principais
usuarios	e-mail único; perfil ENUM(admin, atendente); senha_hash bcrypt
clientes	CPF único (validado também na API)
veiculos	placa única; FK cliente_id com ON DELETE RESTRICT; CHECK no ano
servicos	descrição única; valor DECIMAL(10,2) ≥ 0
agendamentos	FKs para veículo, serviço e usuário; status ENUM; sem conflito de horário por veículo (regra na API)

4. Arquitetura e código-fonte

Arquivo	Entregável	Responsabilidade
backend/src/routes/auth.js, middleware/auth.js, frontend/src/pages/Login.jsx, AuthContext.jsx	4 - Login	Login, registro, logout, sessão por cookie JWT, rate limit, proteção de rotas no servidor e no cliente
backend/src/server.js, frontend/src/App.jsx	5 - Principal	Servidor Express (helmet, CORS, cookies, rotas protegidas); rotas e layout do React
backend/src/routes/clientes.js, frontend/src/pages/Clientes.jsx	6 - Clientes	CRUD de clientes e validação de CPF
backend/src/routes/veiculos.js, frontend/src/pages/Veiculos.jsx	7 - Veículos	CRUD de veículos e validação de placa
backend/src/routes/agendamentos.js, frontend/src/pages/Agendamentos.jsx	8 - Agendamentos	CRUD de agendamentos, conflito de horário, resumo do painel
frontend/src/components/CrudPage.jsx	apoio	Componente de CRUD reutilizado pelas três telas

4.1 Endpoints da API

Método e rota	Descrição	Acesso
POST /api/auth/login	Autentica e grava cookie de sessão	Público
POST /api/auth/registro	Cria usuário (perfil atendente)	Público
POST /api/auth/logout	Encerra a sessão	Público
GET /api/auth/me	Dados do usuário logado	Autenticado
GET/POST /api/clientes, PUT/DELETE /api/clientes/:id	CRUD de clientes	Autenticado
GET/POST /api/veiculos, PUT/DELETE /api/veiculos/:id	CRUD de veículos	Autenticado
GET /api/servicos	Catálogo de serviços	Autenticado
GET/POST /api/agendamentos, PUT/DELETE /api/agendamentos/:id	CRUD de agendamentos (GET aceita ?status=)	Autenticado
GET /api/agendamentos/resumo	Totais e próximos serviços do painel	Autenticado

5. Segurança

Ameaça	Contramedida
Roubo de senhas	Hash bcrypt (custo 10); nenhuma senha em texto puro
Força bruta	express-rate-limit: 10 tentativas por 15 minutos por IP nas rotas de login/registro
Enumeração de usuários	Mesma mensagem de erro e comparação bcrypt mesmo para e-mail inexistente
Roubo de token (XSS)	JWT em cookie httpOnly; React escapa HTML por padrão; helmet define cabeçalhos seguros
CSRF	Cookie SameSite=Strict e API somente JSON; CORS restrito à origem do .env
SQL Injection	Somente prepared statements (mysql2 execute)
Vazamento de segredos	Credenciais e JWT_SECRET no .env (ignorado pelo git); servidor recusa iniciar sem eles
Acesso sem login	Middleware autenticar em todas as rotas de negócio; páginas protegidas no React

6. Variáveis de ambiente (.env)

Arquivo	Variável	Função
backend/.env	PORT, CORS_ORIGIN	Porta da API e origem do front-end permitida
backend/.env	DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	Conexão com o MySQL
backend/.env	JWT_SECRET, JWT_EXPIRES_IN	Assinatura e validade do token (segredo com ao menos 32 caracteres)
frontend/.env	VITE_API_URL, VITE_PROXY_TARGET	URL base da API e destino do proxy em desenvolvimento

7. Instalação e execução

```
# 1) Banco
mysql -u root -p < treino_db.sql

# 2) API
cd sistema/backend
cp .env.example .env   # edite DB_PASSWORD e JWT_SECRET
npm install
npm start              # http://localhost:3001

# 3) Front-end
cd sistema/frontend
cp .env.example .env
npm install
npm run dev            # http://localhost:5173
```

Usuários de teste

E-mail	Senha	Perfil
admin@treino.com	Admin@123	admin
atendente@treino.com	Atend@123	atendente

8. Testes realizados

#	Cenário	Resultado
1	Acessar rota protegida sem login	401 Não autenticado
2	Login com senha errada / tentativa de SQL Injection	401 E-mail ou senha inválidos
3	Mais de 10 tentativas de login seguidas	429 Muitas tentativas
4	Cliente com CPF inválido / duplicado	400 / 409
5	Veículo com placa inválida	400
6	Agendar mesmo veículo e horário duas vezes	409 Conflito
7	Excluir cliente que possui veículos	409 Registros vinculados
8	Logout e nova consulta ao /me	401
9	Build de produção do React e integração Vite → API → MySQL	Sucesso